Backpropagation Through Time

Teorema de derivadas ordenadas y cálculo del gradiente

Emiliano Rodríguez Arango Mario Pacheco López Danny Samuel Martínez Lobo

Universidad El Bosque

29 de octubre de 2026

Origen y motivación de la retropropagación

El entrenamiento de una red neuronal requiere determinar cómo debe modificarse cada peso para reducir el error de sus predicciones.

El problema del cálculo del gradiente

En una red con varias capas, un peso puede influir sobre el error a través de numerosos cálculos intermedios. Es necesario considerar todas esas dependencias para obtener su derivada.

La retropropagación permite calcular esas derivadas de manera organizada, aplicando la regla de la cadena desde las salidas hacia las neuronas anteriores.

El trabajo de **Paul J. Werbos** contribuyó a formular este procedimiento mediante las **derivadas ordenadas** y a extenderlo a sistemas con dependencias temporales.

Dependencias temporales y memoria

En una red sin conexiones temporales, la predicción para una observación se calcula a partir de sus entradas actuales, sin utilizar directamente los estados internos de instantes anteriores.

En problemas secuenciales, como el reconocimiento del habla o el seguimiento de objetos móviles, la información de instantes anteriores puede ser necesaria para interpretar la observación presente.

Incorporación de memoria

Las conexiones recurrentes permiten que el cálculo en el instante t utilice estados internos obtenidos en $t-1$, $t-2$ u otros instantes anteriores.

Así, los estados calculados en un momento pueden influir en las predicciones y los errores de momentos posteriores.

De Backpropagation a Backpropagation Through Time

En la **retropropagación convencional**, las derivadas del error se calculan en sentido inverso al orden de las operaciones de la red, para una observación determinada.

En la **retropropagación a través del tiempo** (BPTT), el cálculo debe considerar además que los estados de un instante intervienen en los cálculos de instantes posteriores.

Diferencia fundamental

El gradiente incorpora tanto las dependencias entre neuronas como las dependencias entre instantes de tiempo.

Para obtenerlo, primero se calculan los estados de la secuencia y luego se propagan las derivadas desde los instantes posteriores hacia los anteriores.

¿Cuál es el problema?

Una red neuronal realiza una sucesión de cálculos:



¿Cuál es el problema?

Una red neuronal realiza una sucesión de cálculos:



Pregunta

¿Cómo cambia el error cuando modificamos ligeramente uno de los pesos W de la red?

¿Cuál es el problema?

Una red neuronal realiza una sucesión de cálculos:



Pregunta

¿Cómo cambia el error cuando modificamos ligeramente uno de los pesos W de la red?

Por tanto, es una regla de la cadena:

$$\boxed{\frac{\partial E}{\partial W}}$$

El error **respecto al peso**.

La derivada se interpreta como una **medida de sensibilidad**.

$$\boxed{\frac{\partial E}{\partial W} = \text{¿Cuánto cambia el error si cambio ligeramente } W\text{?}}$$

Si:

La derivada se interpreta como una **medida de sensibilidad**.

$$\frac{\partial E}{\partial W} = \text{¿Cuánto cambia el error si cambio ligeramente } W\text{?}$$

Si:

Caso 1

$$\frac{\partial E}{\partial W} > 0$$

Aumentar W aumenta el error.

Caso 2

$$\frac{\partial E}{\partial W} < 0$$

Aumentar W disminuye el error.

Caso 3

$$\frac{\partial E}{\partial W} \approx 0$$

El peso tiene poco efecto sobre el error.

La derivada se interpreta como una **medida de sensibilidad**.

$$\frac{\partial E}{\partial W} = \text{¿Cuánto cambia el error si cambio ligeramente } W\text{?}$$

Si:

Caso 1

$$\frac{\partial E}{\partial W} > 0$$

Aumentar W aumenta el error.

Caso 2

$$\frac{\partial E}{\partial W} < 0$$

Aumentar W disminuye el error.

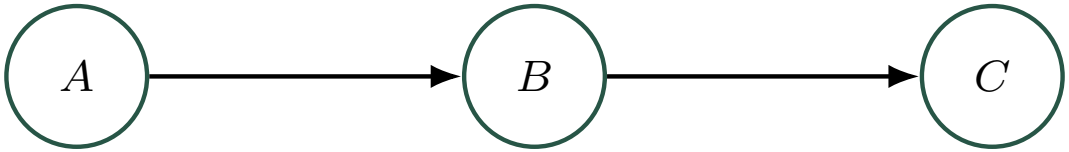
Caso 3

$$\frac{\partial E}{\partial W} \approx 0$$

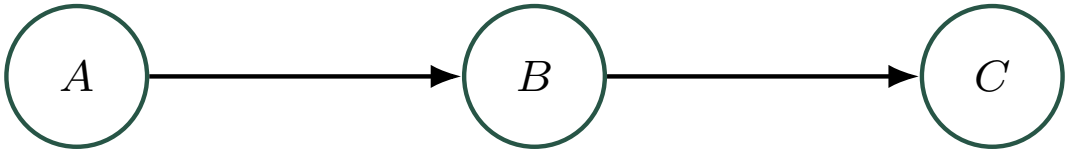
El peso tiene poco efecto sobre el error.

La derivada sirve para modificar los pesos.

Suponga la siguiente relación:



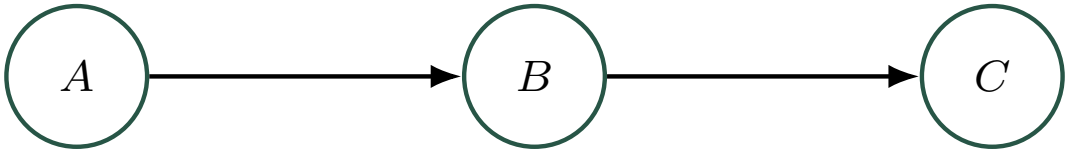
Suponga la siguiente relación:



¿Cómo funciona la regla de la cadena?

$$\boxed{\frac{dC}{dA} = \frac{dC}{dB}\frac{dB}{dA}}$$

Suponga la siguiente relación:



¿Cómo funciona la regla de la cadena?

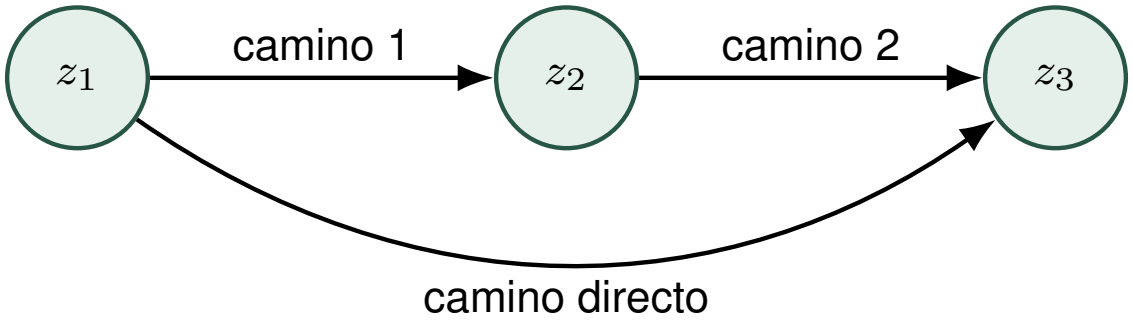
$$\boxed{\frac{dC}{dA} = \frac{dC}{dB}\frac{dB}{dA}}$$

Recuerde

Backpropagation busca hacer uso de la regla de cadena para medir esos impactos.

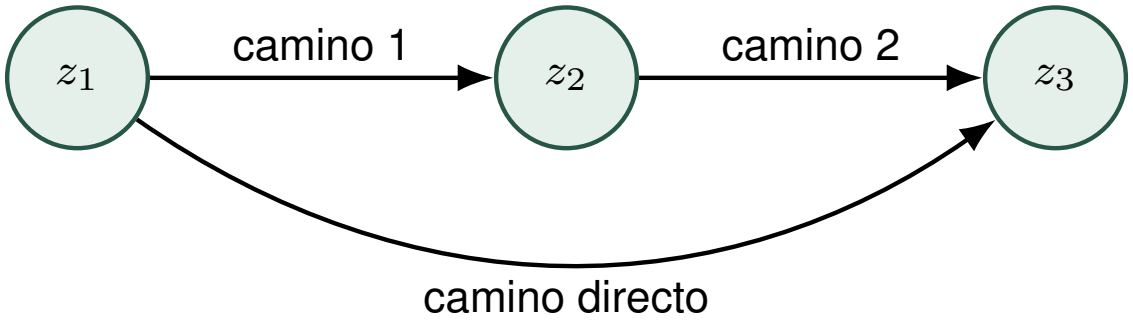
¿Cuál es el problema en las redes? existen múltiples caminos

En una red, una variable puede afectar el resultado por múltiples caminos.



¿Cuál es el problema en las redes? existen múltiples caminos

En una red, una variable puede afectar el resultado por múltiples caminos.

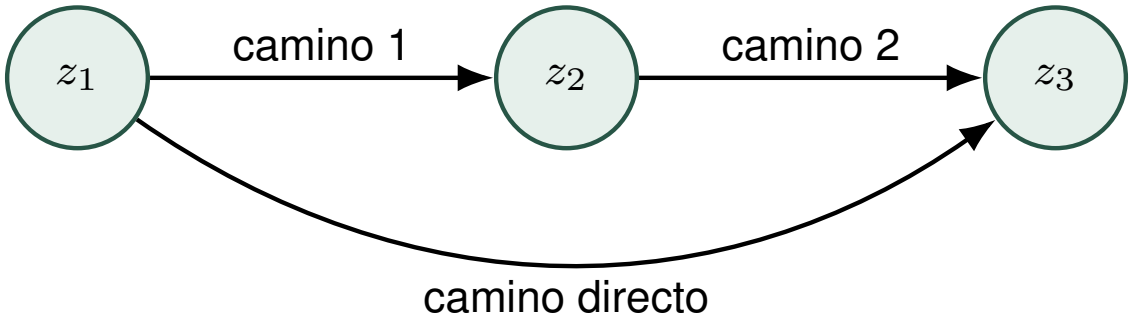


luego, al cambiar z_1:

- se afecta z_3;

¿Cuál es el problema en las redes? existen múltiples caminos

En una red, una variable puede afectar el resultado por múltiples caminos.

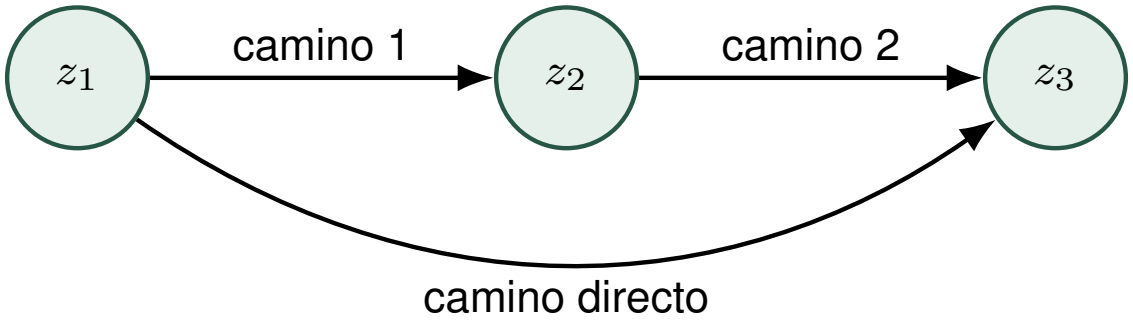


luego, al cambiar z_1:

- se afecta z_3;
- se afecta z_2;

¿Cuál es el problema en las redes? existen múltiples caminos

En una red, una variable puede afectar el resultado por múltiples caminos.

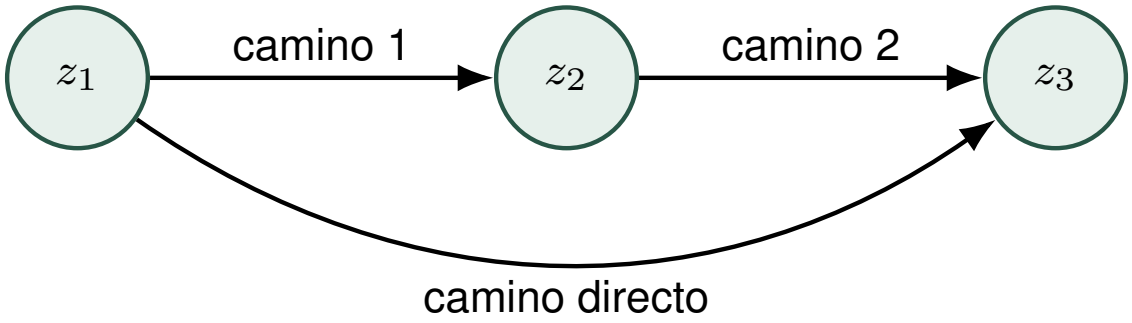


luego, al cambiar z_1:

- se afecta z_3;
- se afecta z_2;
- el cambio de z_2 influye posteriormente a z_3.

¿Cuál es el problema en las redes? existen múltiples caminos

En una red, una variable puede afectar el resultado por múltiples caminos.



luego, al cambiar z_1:

- se afecta z_3;
- se afecta z_2;
- el cambio de z_2 influye posteriormente a z_3.

¿Cuál es el inconveniente?

La derivada parcial clásica mide **efecto directo**, pero se requiere conocer el **efecto total**.

Teorema de derivadas ordenadas

Werbos introduce una forma de la regla de la cadena para sistemas con un **orden causal de cálculo**:

$$z_1, z_2, \ldots, z_n, \text{TARGET}$$

Teorema de derivadas ordenadas

Werbos introduce una forma de la regla de la cadena para sistemas con un **orden causal de cálculo**:

$$z_1, z_2, \ldots, z_n, \text{TARGET}$$

La derivada se puede escribir como:

$$\boxed{\frac{\partial^+ \text{TARGET}}{\partial z_i} = \frac{\partial \text{TARGET}}{\partial z_i} + \sum_{j>i} \frac{\partial^+ \text{TARGET}}{\partial z_j} \frac{\partial z_j}{\partial z_i}}$$

Teorema de derivadas ordenadas

Werbos introduce una forma de la regla de la cadena para sistemas con un **orden causal de cálculo**:

$$z_1, z_2, \ldots, z_n, \text{TARGET}$$

La derivada se puede escribir como:

$$\boxed{\frac{\partial^+ \text{TARGET}}{\partial z_i} = \frac{\partial \text{TARGET}}{\partial z_i} + \sum_{j>i} \frac{\partial^+ \text{TARGET}}{\partial z_j} \frac{\partial z_j}{\partial z_i}}$$

donde:

Primera parte

$$\frac{\partial \text{TARGET}}{\partial z_i}$$

Representa los efectos **directos**.

Segunda parte

$$\sum_{j>i} \frac{\partial^+ \text{TARGET}}{\partial z_j} \frac{\partial z_j}{\partial z_i}$$

Es una suma de los **efectos indirectos**.

Ejemplo del Autor

Sea el sistema:

$$\boxed{z_2 = 4z_1}$$

$$\boxed{z_3 = 3z_1 + 5z_2}$$

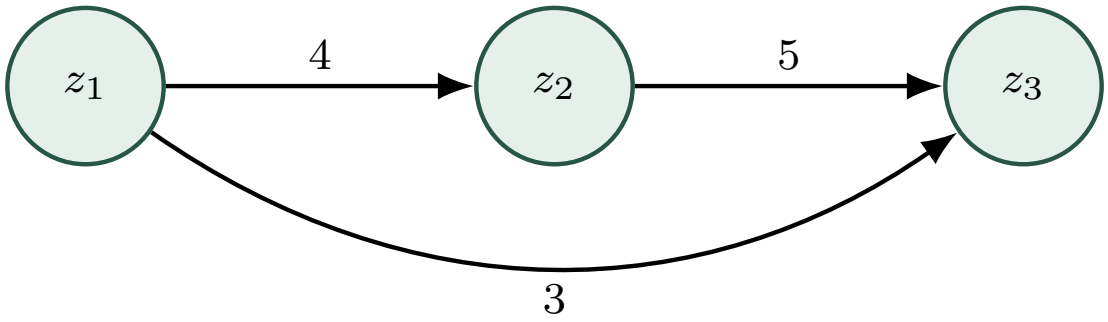
Ejemplo del Autor

Sea el sistema:

$$\boxed{z_2 = 4z_1}$$

$$\boxed{z_3 = 3z_1 + 5z_2}$$

Las derivadas son:



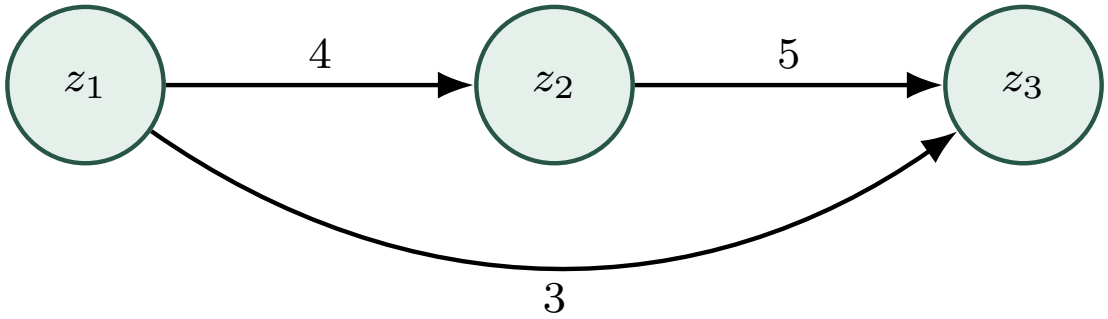
Ejemplo del Autor

Sea el sistema:

$$\boxed{z_2 = 4z_1}$$

$$\boxed{z_3 = 3z_1 + 5z_2}$$

Las derivadas son:



Tenemos dos caminos desde z_1 hasta z_3:

$$z_1 \longrightarrow z_3$$

y

$$z_1 \longrightarrow z_2 \longrightarrow z_3$$

Teorema de derivadas ordenadas

$$\frac{\partial^+ z_3}{\partial z_1} = \underbrace{3}_{\text{efecto directo}} + \underbrace{(5)(4)}_{\text{efecto indirecto}}$$

Teorema de derivadas ordenadas

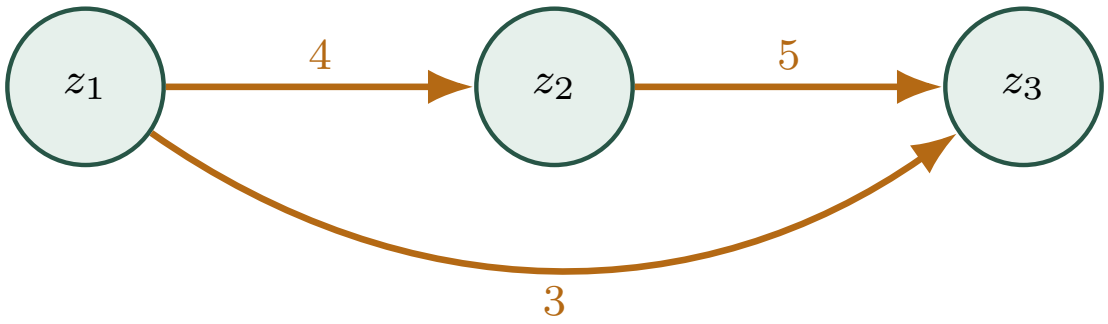
$$\frac{\partial^+ z_3}{\partial z_1} = \underbrace{3}_{\text{efecto directo}} + \underbrace{(5)(4)}_{\text{efecto indirecto}}$$

$$\boxed{\frac{\partial^+ z_3}{\partial z_1} = 3 + 20 = 23}$$

Teorema de derivadas ordenadas

$$\frac{\partial^+ z_3}{\partial z_1} = \underbrace{3}_{\text{efecto directo}} + \underbrace{(5)(4)}_{\text{efecto indirecto}}$$

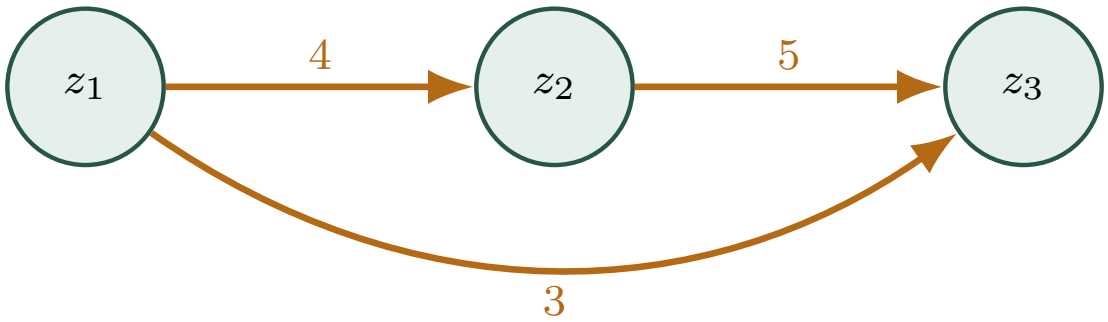
$$\boxed{\frac{\partial^+ z_3}{\partial z_1} = 3 + 20 = 23}$$



Teorema de derivadas ordenadas

$$\frac{\partial^+ z_3}{\partial z_1} = \underbrace{3}_{\text{efecto directo}} + \underbrace{(5)(4)}_{\text{efecto indirecto}}$$

$$\boxed{\frac{\partial^+ z_3}{\partial z_1} = 3 + 20 = 23}$$



Warning

Si z_1 aumenta una unidad, z_3 aumenta 23 unidades, porque deben considerarse **todos los caminos**.

¿Cómo se aplica en las redes?

Una neurona realiza dos operaciones básicas:

$$net_i = \sum_j W_{ij} x_j$$

¿Cómo se aplica en las redes?

Una neurona realiza dos operaciones básicas:

$$net_i = \sum_j W_{ij} x_j$$

y posteriormente:

$$x_i = s(net_i)$$

donde s es la función de activación.

¿Cómo se aplica en las redes?

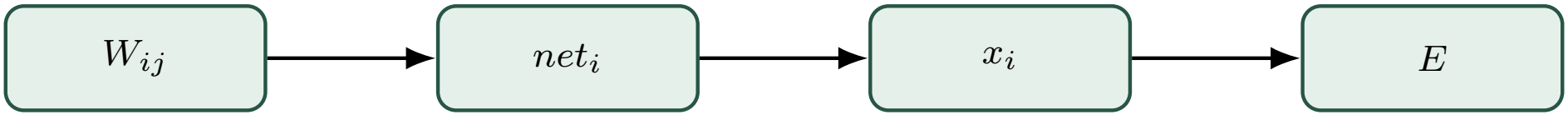
Una neurona realiza dos operaciones básicas:

$$net_i = \sum_j W_{ij} x_j$$

y posteriormente:

$$x_i = s(net_i)$$

donde s es la función de activación.



¿Cómo se aplica en las redes?

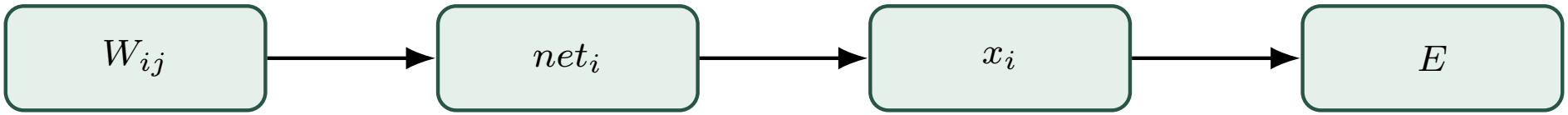
Una neurona realiza dos operaciones básicas:

$$net_i = \sum_j W_{ij} x_j$$

y posteriormente:

$$x_i = s(net_i)$$

donde s es la función de activación.



Por la regla de la cadena:

$$\boxed{\frac{\partial E}{\partial W_{ij}} = \frac{\partial E}{\partial x_i} \frac{\partial x_i}{\partial net_i} \frac{\partial net_i}{\partial W_{ij}}}$$

La derivada de la función sigmoide

Werbos utiliza la función sigmoide:

$$s(z) = \frac{1}{1 + e^{-z}}$$

La derivada de la función sigmoide

Werbos utiliza la función sigmoide:

$$s(z) = \frac{1}{1 + e^{-z}}$$

Su derivada puede expresarse de manera muy conveniente:

$$\boxed{s'(z) = s(z)\,[1 - s(z)]}$$

La derivada de la función sigmoide

Werbos utiliza la función sigmoide:

$$s(z) = \frac{1}{1 + e^{-z}}$$

Su derivada puede expresarse de manera muy conveniente:

$$\boxed{s'(z) = s(z)\left[1 - s(z)\right]}$$

Como:

$$x_i = s(net_i)$$

entonces:

$$\boxed{\frac{\partial x_i}{\partial net_i} = x_i(1 - x_i)}$$

La derivada de la función sigmoide

Werbos utiliza la función sigmoide:

$$s(z) = \frac{1}{1 + e^{-z}}$$

Su derivada puede expresarse de manera muy conveniente:

$$\boxed{s'(z) = s(z)\,[1 - s(z)]}$$

Como:

$$x_i = s(net_i)$$

entonces:

$$\boxed{\frac{\partial x_i}{\partial net_i} = x_i(1 - x_i)}$$

Esto se interpreta como:

Esta derivada mide qué tan sensible es la activación x_i frente a un pequeño cambio en la entrada neta net_i de la neurona.

El flujo hacia atrás: Backward Pass

Durante el cálculo normal la información fluye hacia adelante:



El flujo hacia atrás: Backward Pass

Durante el cálculo normal la información fluye hacia adelante:



Las derivadas necesarias para calcular el gradiente de los pesos se obtienen en sentido contrario:



El flujo hacia atrás: Backward Pass

Durante el cálculo normal la información fluye hacia adelante:



Las derivadas necesarias para calcular el gradiente de los pesos se obtienen en sentido contrario:



¿Por qué “backpropagation”?

Porque la información sobre el error se **propaga hacia atrás** para calcular cuánto contribuyó cada variable y cada peso al error final.

El feedback de una neurona

Werbos representa la derivada ordenada mediante el prefijo F de *feedback*.

$$\boxed{F_{x_i} \equiv \frac{\partial^+ E}{\partial x_i}}$$

El feedback de una neurona

Werbos representa la derivada ordenada mediante el prefijo F de *feedback*.

$$\boxed{F_{x_i} \equiv \frac{\partial^+ E}{\partial x_i}}$$

Para una neurona:

$$F_{net_i} = s'(net_i) F_{x_i}$$

El feedback de una neurona

Werbos representa la derivada ordenada mediante el prefijo F de *feedback*.

$$\boxed{F_{x_i} \equiv \frac{\partial^+ E}{\partial x_i}}$$

Para una neurona:

$$F_{net_i} = s'(net_i) F_{x_i}$$

Utilizando la derivada de la sigmoide:

$$\boxed{F_{net_i} = x_i(1 - x_i) F_{x_i}}$$

El feedback de una neurona

Werbos representa la derivada ordenada mediante el prefijo F de *feedback*.

$$\boxed{F_{x_i} \equiv \frac{\partial^+ E}{\partial x_i}}$$

Para una neurona:

$$F_{net_i} = s'(net_i) F_{x_i}$$

Utilizando la derivada de la sigmoide:

$$\boxed{F_{net_i} = x_i(1 - x_i) F_{x_i}}$$

¿qué significa?

F_{net_i} representa cuánta “responsabilidad” tiene la entrada neta de la neurona sobre el error final.

Gradiente respecto a los pesos

Como:

$$net_i = \sum_j W_{ij} x_j$$

tenemos:

$$\frac{\partial net_i}{\partial W_{ij}} = x_j$$

Gradiente respecto a los pesos

Como:

$$net_i = \sum_j W_{ij} x_j$$

tenemos:

$$\frac{\partial net_i}{\partial W_{ij}} = x_j$$

Entonces:

$$\boxed{\frac{\partial E}{\partial W_{ij}} = F_{net_i} x_j}$$

Gradiente respecto a los pesos

Como:

$$net_i = \sum_j W_{ij} x_j$$

tenemos:

$$\frac{\partial net_i}{\partial W_{ij}} = x_j$$

Entonces:

$$\boxed{\frac{\partial E}{\partial W_{ij}} = F_{net_i} x_j}$$

Para múltiples observaciones:

$$\boxed{F_{W_{ij}} = \sum_t F_{net_i}(t) x_j(t)}$$

Gradiente respecto a los pesos

Como:

$$net_i = \sum_j W_{ij} x_j$$

tenemos:

$$\frac{\partial net_i}{\partial W_{ij}} = x_j$$

Entonces:

$$\boxed{\frac{\partial E}{\partial W_{ij}} = F_{net_i} x_j}$$

Para múltiples observaciones:

$$\boxed{F_{W_{ij}} = \sum_t F_{net_i}(t) x_j(t)}$$

Por lo que el gradiente de un peso es el producto entre:

1. la señal que entró por esa conexión, x_j;
2. la señal de error que regresó, F_{net_i}.

Actualización de los pesos

Una vez calculado el gradiente:

$$\frac{\partial E}{\partial W_{ij}}$$

el peso se modifica mediante descenso de gradiente:

$$\boxed{W_{ij}^{nuevo} = W_{ij}^{viejo} - \eta \frac{\partial E}{\partial W_{ij}}}$$

donde:

$$\eta = \text{learning rate}$$

Actualización de los pesos

Una vez calculado el gradiente:

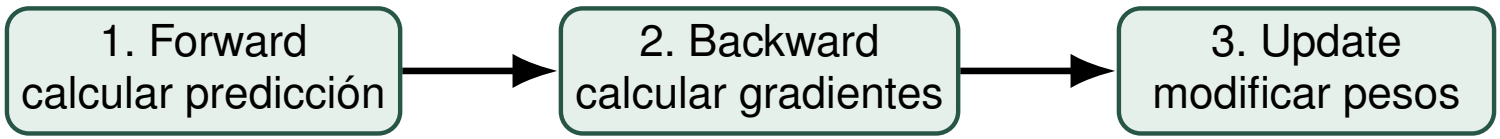
$$\frac{\partial E}{\partial W_{ij}}$$

el peso se modifica mediante descenso de gradiente:

$$\boxed{W_{ij}^{nuevo} = W_{ij}^{viejo} - \eta \frac{\partial E}{\partial W_{ij}}}$$

donde:

$$\eta = \text{learning rate}$$



Actualización de los pesos

Una vez calculado el gradiente:

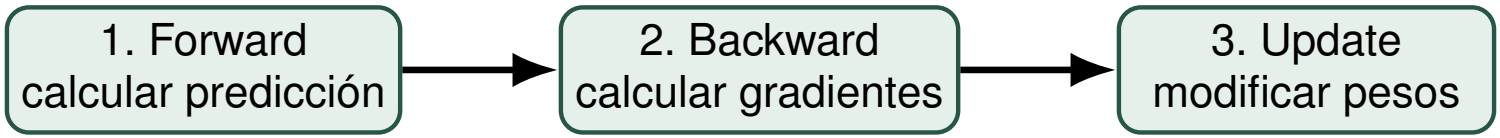
$$\frac{\partial E}{\partial W_{ij}}$$

el peso se modifica mediante descenso de gradiente:

$$\boxed{W_{ij}^{nuevo} = W_{ij}^{viejo} - \eta \frac{\partial E}{\partial W_{ij}}}$$

donde:

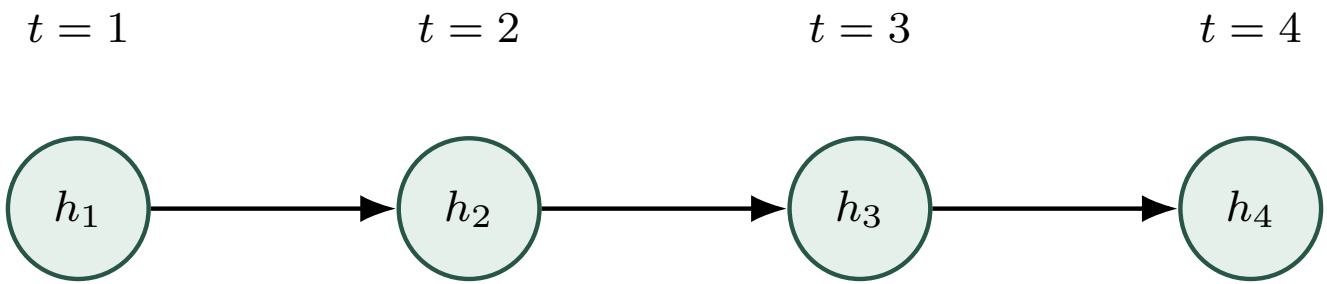
$$\eta = \text{learning rate}$$



El procedimiento se repite hasta reducir el error.

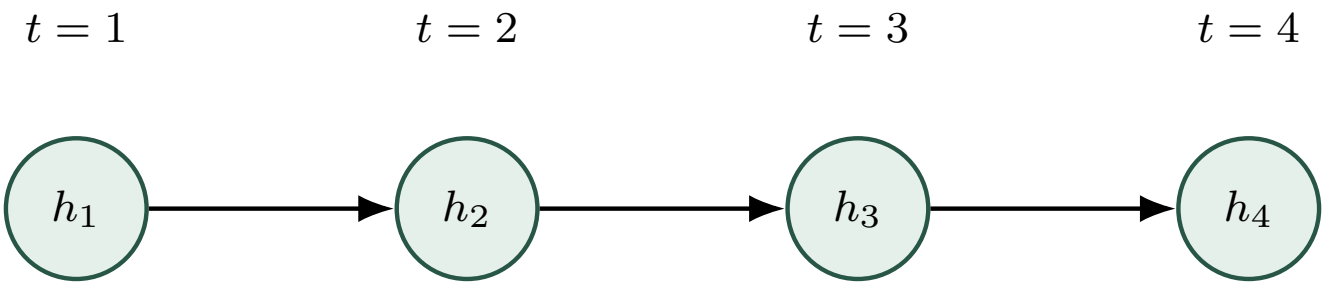
¿Qué sucede cuando aparece el tiempo?

En una red recurrente, el estado actual depende de estados anteriores.



¿Qué sucede cuando aparece el tiempo?

En una red recurrente, el estado actual depende de estados anteriores.



Por tanto:

$$h_4 \text{ depende de } h_3,$$

pero:

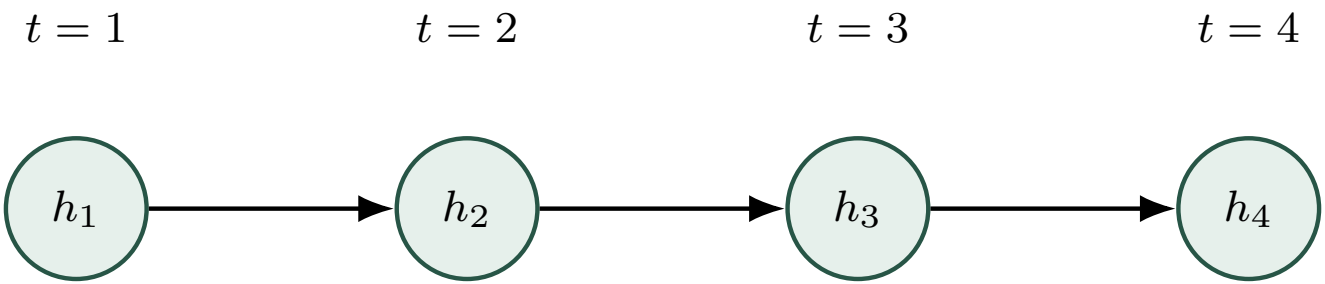
$$h_3 \text{ depende de } h_2,$$

y:

$$h_2 \text{ depende de } h_1.$$

¿Qué sucede cuando aparece el tiempo?

En una red recurrente, el estado actual depende de estados anteriores.



Por tanto:

$$h_4 \text{ depende de } h_3,$$

pero:

$$h_3 \text{ depende de } h_2,$$

y:

$$h_2 \text{ depende de } h_1.$$

Consecuencia

Una decisión tomada en $t = 1$ puede afectar un error que sólo aparece mucho después.

Supongamos que el error final depende de h_3:

$$h_1 \longrightarrow h_2 \longrightarrow h_3 \longrightarrow E$$

Supongamos que el error final depende de h_3:

$$h_1 \longrightarrow h_2 \longrightarrow h_3 \longrightarrow E$$

Por la regla de la cadena para conocer el efecto de h_1

$$\boxed{\frac{\partial E}{\partial h_1} = \frac{\partial E}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}}$$

Supongamos que el error final depende de h_3:

$$h_1 \longrightarrow h_2 \longrightarrow h_3 \longrightarrow E$$

Por la regla de la cadena para conocer el efecto de h_1

$$\boxed{\frac{\partial E}{\partial h_1} = \frac{\partial E}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}}$$

Supongamos que el error final depende de h_3:

$$h_1 \longrightarrow h_2 \longrightarrow h_3 \longrightarrow E$$

Por la regla de la cadena para conocer el efecto de h_1

$$\boxed{\frac{\partial E}{\partial h_1} = \frac{\partial E}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}}$$

¿cuál es la diferencia?

En backpropagation la cadena debe atravesar las **capas**.
En BPTT también debe atravesar los **instantes de tiempo**.

La ecuación del BPTT

En el modelo de Werbos una neurona puede recibir efectos:

- del instante actual;
- del instante siguiente;
- incluso de dos instantes posteriores.

La ecuación del BPTT

En el modelo de Werbos una neurona puede recibir efectos:

- del instante actual;
- del instante siguiente;
- incluso de dos instantes posteriores.

Por tanto,

$$\boxed{F_{x_i}(t) = \text{impacto actual} + \text{impacto desde } t+1 + \text{impacto desde } t+2}$$

La ecuación del BPTT

En el modelo de Werbos una neurona puede recibir efectos:

- del instante actual;
- del instante siguiente;
- incluso de dos instantes posteriores.

Por tanto,

$$\boxed{F_{x_i}(t) = \text{impacto actual} + \text{impacto desde } t+1 + \text{impacto desde } t+2}$$

Una forma equivalente al caso desarrollado por Werbos es:

$$F_{x_i}(t) = F_{\widehat{Y}_{i-N}}(t) + \sum_j W_{ji} F_{net_j}(t) + \sum_j W'_{ji} F_{net_j}(t+1) + \sum_j W''_{ji} F_{net_j}(t+2)$$

La ecuación del BPTT

En el modelo de Werbos una neurona puede recibir efectos:

- del instante actual;
- del instante siguiente;
- incluso de dos instantes posteriores.

Por tanto,

$$\boxed{F_{x_i}(t) = \text{impacto actual} + \text{impacto desde } t+1 + \text{impacto desde } t+2}$$

Una forma equivalente al caso desarrollado por Werbos es:

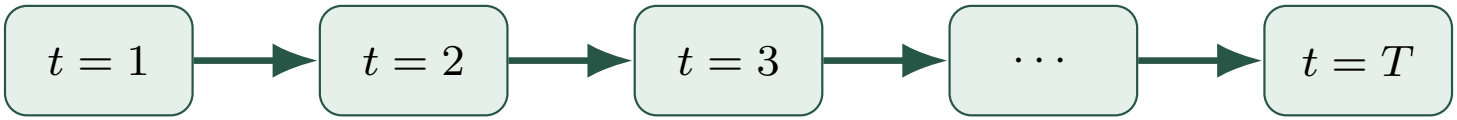
$$F_{x_i}(t) = F_{\widehat{Y}_{i-N}}(t) + \sum_j W_{ji} F_{net_j}(t) + \sum_j W'_{ji} F_{net_j}(t+1) + \sum_j W''_{ji} F_{net_j}(t+2)$$

Cuidado

Para calcular $F_{x_i}(t)$ se necesita conocer las derivadas correspondientes a tiempos futuros. Por eso se calculan las derivadas **desde T hacia 1**.

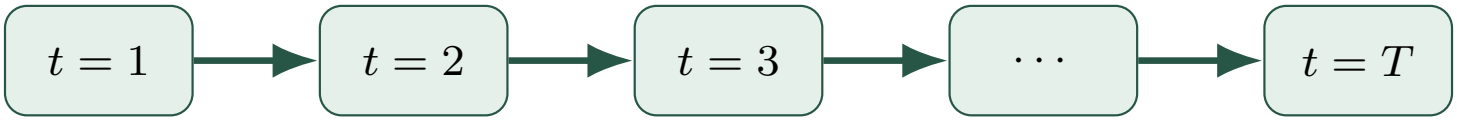
¿Por qué se debe retroceder en el tiempo?

Primero se ejecuta la red hacia adelante:



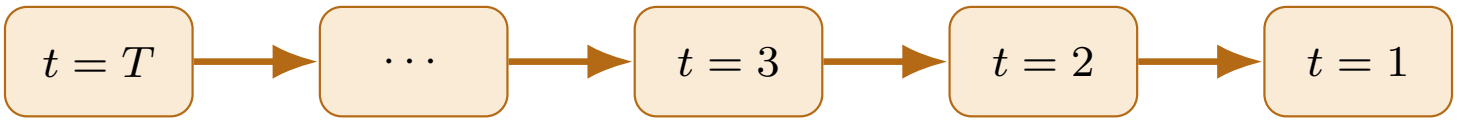
¿Por qué se debe retroceder en el tiempo?

Primero se ejecuta la red hacia adelante:



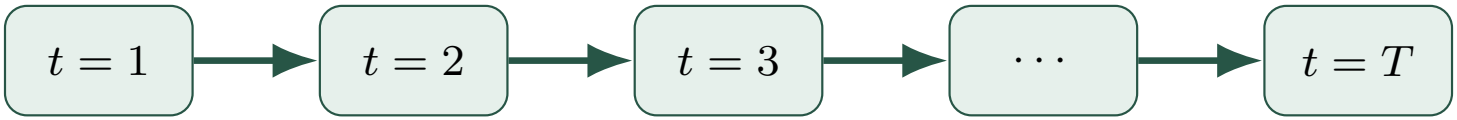
Se guardan las activaciones y estados intermedios.

Después se calculan las derivadas:



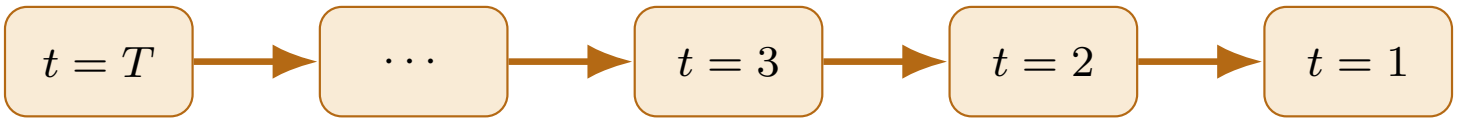
¿Por qué se debe retroceder en el tiempo?

Primero se ejecuta la red hacia adelante:



Se guardan las activaciones y estados intermedios.

Después se calculan las derivadas:



Forward: $1 \rightarrow T$ Backward: $T \rightarrow 1$

Gradiente de las conexiones temporales

Cuando un mismo peso participa repetidamente a lo largo del tiempo, su efecto total es la suma de sus contribuciones.

Gradiente de las conexiones temporales

Cuando un mismo peso participa repetidamente a lo largo del tiempo, su efecto total es la suma de sus contribuciones.
Para una conexión con retardo de un periodo:

$$F_{W'_{ij}} = \sum_{t} F_{net_i}(t+1)x_j(t)$$

Gradiente de las conexiones temporales

Cuando un mismo peso participa repetidamente a lo largo del tiempo, su efecto total es la suma de sus contribuciones.
Para una conexión con retardo de un periodo:

$$F_{W'_{ij}} = \sum_t F_{net_i}(t+1)x_j(t)$$

Para una conexión con retardo de dos periodos:

$$F_{W''_{ij}} = \sum_t F_{net_i}(t+2)x_j(t)$$

Gradiente de las conexiones temporales

Cuando un mismo peso participa repetidamente a lo largo del tiempo, su efecto total es la suma de sus contribuciones.
Para una conexión con retardo de un periodo:

$$F_{W'_{ij}} = \sum_t F_{net_i}(t+1)x_j(t)$$

Para una conexión con retardo de dos periodos:

$$F_{W''_{ij}} = \sum_t F_{net_i}(t+2)x_j(t)$$

Interpretación

El peso temporal se utiliza muchas veces durante la secuencia.

Gradiente de las conexiones temporales

Cuando un mismo peso participa repetidamente a lo largo del tiempo, su efecto total es la suma de sus contribuciones.
Para una conexión con retardo de un periodo:

$$F_{W'_{ij}} = \sum_t F_{net_i}(t+1)x_j(t)$$

Para una conexión con retardo de dos periodos:

$$F_{W''_{ij}} = \sum_t F_{net_i}(t+2)x_j(t)$$

Interpretación

El peso temporal se utiliza muchas veces durante la secuencia. Por eso su gradiente final acumula **el error en todos los tiempos** en los que fue utilizado.

Resumen del algoritmo BPTT

1. **Forward Pass**
   Ejecutar la red desde:

$$t = 1 \rightarrow T$$

   guardando:

$$x(t), \qquad net(t)$$

Resumen del algoritmo BPTT

1. **Forward Pass**
   Ejecutar la red desde:

$$t = 1 \rightarrow T$$

   guardando:

$$x(t), \qquad net(t)$$

2. **Backward Pass**
   Recorrer:

$$t = T \rightarrow 1$$

   aplicando la regla de la cadena y acumulando gradientes.

Resumen del algoritmo BPTT

1. **Forward Pass**
   Ejecutar la red desde:

$$t = 1 \rightarrow T$$

   guardando:

$$x(t), \qquad net(t)$$

2. **Backward Pass**
   Recorrer:

$$t = T \rightarrow 1$$

   aplicando la regla de la cadena y acumulando gradientes.

3. **Actualización**
   Modificar los pesos mediante:

$$W^{nuevo} = W^{viejo} - \eta \nabla_W E$$

Resumen del algoritmo BPTT

1. **Forward Pass**
   Ejecutar la red desde:

$$t = 1 \rightarrow T$$

   guardando:

$$x(t), \qquad net(t)$$

2. **Backward Pass**
   Recorrer:

$$t = T \rightarrow 1$$

   aplicando la regla de la cadena y acumulando gradientes.

3. **Actualización**
   Modificar los pesos mediante:

$$W^{nuevo} = W^{viejo} - \eta \nabla_W E$$

En Resumen

BPTT distribuye hacia atrás la responsabilidad del error, atravesando simultáneamente la estructura de la red y su historia temporal.

Procedimiento NET: cálculo hacia adelante

NET: entradas y resultados

```
NET(X(t), W):
    para i = 1,...,m:
        x[i](t) <- X[i](t)

    para i = m+1,...,N+n:
        net <- 0
        para j = 1,...,i-1:
            net <- net + W[i,j] * x[j](t)
        x[i](t) <- 1 / (1 + exp(-net))

    para k = 1,...,n:
        Yhat[k](t) <- x[N+k](t)

    devolver x(t), Yhat(t)
```

Se incorporan primero las entradas; después se calculan las excitaciones y las activaciones en orden ascendente. Las últimas n activaciones son las predicciones.

Procedimiento F_NET: retropropagación

F_NET: entradas y resultados

```
F_NET(F_Yhat(t), W, x(t)):
    para i = 1,...,N:
        F_x[i](t) <- 0

    para k = 1,...,n:
        F_x[N+k](t) <- F_Yhat[k](t)

    para i = N+n,...,m+1 (orden descendente):
        para j = i+1,...,N+n:
            F_x[i](t) <- F_x[i](t) + W[j,i] * F_net[j](t)
        F_net[i](t) <- F_x[i](t) * x[i](t) * (1-x[i](t))
        para j = 1,...,i-1:
            F_W[i,j](t) <- F_net[i](t) * x[j](t)

    devolver F_W(t)
```

El recorrido descendente calcula las derivadas ordenadas respecto a las salidas y excitaciones, y después las derivadas respecto a cada peso.

Entrenamiento por observación

Entrenamiento por observación

```
repetir hasta cumplir el criterio de parada:
    para t = 1,...,T:
        x(t), Yhat(t) <- NET(X(t), W)
        para k = 1,...,n:
            F_Yhat[k](t) <- Yhat[k](t) - Y[k](t)
        F_W(t) <- F_NET(F_Yhat(t), W, x(t))
        para i = m+1,...,N+n:
            para j = 1,...,i-1:
                W[i,j] <- W[i,j] - eta * F_W[i,j](t)
```

Los pesos se modifican después de cada observación. La tasa η determina el tamaño de la actualización.

Procedimiento NET2: dos rezagos

NET2: entradas y resultados

```
NET2(X(t), W, W', W'', x(t-2), x(t-1)):
    para i = 1,...,m:
        x[i](t) <- X[i](t)

    para i = m+1,...,N+n:
        net <- 0
        para j = 1,...,i-1:
            net <- net + W[i,j] * x[j](t)
        para j = 1,...,N+n:
            net <- net + W'[i,j] * x[j](t-1)
            net <- net + W''[i,j] * x[j](t-2)
        x[i](t) <- 1 / (1 + exp(-net))

    para k = 1,...,n:
        Yhat[k](t) <- x[N+k](t)

    devolver x(t), Yhat(t)
```

W contiene las conexiones actuales; W' y W'' contienen las conexiones con uno y dos rezagos, respectivamente.

Procedimiento F_NET2: derivadas temporales

F_NET2: entradas y resultados

```
F_NET2(t, F_Yhat, W, W', W'', x, F_net, G_W, G_W1, G_W2):
    para i = 1,...,N:
        F_x[i](t) <- 0
    para k = 1,...,n:
        F_x[N+k](t) <- F_Yhat[k]

    para i = N+n,...,m+1 (orden descendente):
        para j = i+1,...,N+n:
            F_x[i](t) <- F_x[i](t) + W[j,i] * F_net[j](t)
        para j = m+1,...,N+n:
            F_x[i](t) <- F_x[i](t) + W'[j,i] * F_net[j](t+1)
            F_x[i](t) <- F_x[i](t) + W''[j,i] * F_net[j](t+2)
        F_net[i](t) <- F_x[i](t) * x[i](t) * (1-x[i](t))
        para j = 1,...,i-1:
            G_W[i,j] <- G_W[i,j] + F_net[i](t) * x[j](t)
        para j = 1,...,N+n:
            G_W1[i,j] <- G_W1[i,j] + F_net[i](t) * x[j](t-1)
            G_W2[i,j] <- G_W2[i,j] + F_net[i](t) * x[j](t-2)

    devolver F_net(t), G_W, G_W1, G_W2
```

Las contribuciones de $t+1$ y $t+2$ se incorporan en F_x y los gradientes de las tres matrices se acumulan.

Entrenamiento mediante retropropagación temporal

Entrenamiento temporal

```
repetir hasta cumplir el criterio de parada:
    fijar los estados iniciales x(0) y x(-1)
    para t = 1,...,T:
        x(t), Yhat(t) <- NET2(X(t), W, W', W'',
                              x(t-2), x(t-1))

    inicializar G_W, G_W1 y G_W2 en cero
    inicializar F_net(T+1) y F_net(T+2) en cero

    para t = T,...,1 (orden descendente):
        para k = 1,...,n:
            F_Yhat[k] <- Yhat[k](t) - Y[k](t)
        F_net(t), G_W, G_W1, G_W2 <-
            F_NET2(t, F_Yhat, W, W', W'', x,
                   F_net, G_W, G_W1, G_W2)

    para cada peso que se ajusta:
        W[i,j] <- W[i,j] - eta * G_W[i,j]
        W'[i,j] <- W'[i,j] - eta * G_W1[i,j]
        W''[i,j] <- W''[i,j] - eta * G_W2[i,j]
```

Se calculan los estados de toda la secuencia, se acumulan las derivadas desde T hasta 1 y finalmente se actualizan las tres matrices de pesos.

Referencias

[1] P. J. Werbos.
Backpropagation Through Time: What It Does and How to Do It.
Proceedings of the IEEE, Vol. 78, No. 10, pp. 1550–1560, 1990.

[2] D. E. Rumelhart, G. E. Hinton and R. J. Williams.
Learning representations by back-propagating errors.
Nature, Vol. 323, pp. 533–536, 1986.